

Splunk UBA 5.4.2 Deployment Guide

1. System Requirements
 - Hardware Requirements
 - Operating System Requirements
2. Networking Requirements
 - Static IP Address Configuration
 - Required Ports and Firewall Configuration
 - DNS and Hostname Configuration
3. Pre-Installation Tasks
 - Configure Permissions and Prepare the User
 - Prepare Storage
 - Configure the Environment
4. Installation
 - Obtain and Prepare Installation Package
 - Single Server Installation
 - Distributed Server Installation
5. Post-Installation Configuration
 - Verify Installation
 - Secure Default Account
 - Basic Service Management
6. Configure Splunk UBA
 - Upload Splunk UBA License
 - (Optional) Enable FIPS Compliance
 - SSL Certificate Management
7. Manage User Accounts and Authentication
 - User Account Roles
 - Configure Single Sign-On (SSO)
 - Configure Splunk Platform Authentication
8. Backup and Restore Configuration
 - Configure Automated Incremental Backups
 - Manual Full Backup Procedure
 - Restore Procedure
9. Configure Warm Standby for High Availability
10. Monitor Splunk UBA
 - Health Monitoring
 - Performance Optimization
 - Collect Diagnostic Data
11. Customize Splunk UBA
 - Configure Internal IP Ranges
 - Geo-Location Settings
 - Email Alerts Configuration
 - Customize Anomaly and Threat Parameters
 - Masking PII Information
 - Disable UI Timeout (Optional)
12. Data Source Configuration
 - Required Data Sources
 - Splunk Data Source Configuration
 - SSL Certificate Validation for Data Sources
 - Validate Data Ingestion
13. Send Data from Splunk UBA
 - Send UBA Data to Splunk Enterprise Security
 - Send Threats via Email
 - Send Threats to ServiceNow
14. Security Hardening and Best Practices
 - Secure Default Account
 - Third-Party Security Agents
 - Regular Security Updates
 - Certificate Security
15. Troubleshooting
 - Quick Reference - Common Commands
 - Common Log Locations
16. Advanced Administration
 - Managing UBA Configuration Properties
 - Start and Stop Services
 - Change Service Passwords
 - Change IP Address or Hostname

- 17. Service Management Migration and Non-Privileged Operations
 - Switching from Script-Managed to systemctl-Managed Services
 - Configuring Non-Privileged UBA Instance
 - Migration Best Practices
- 18. Deployment Summary and Next Steps
 - Deployment Validation
 - Post-Deployment Activities
 - Key Resources
 - Emergency Procedures
 - Support Information

Splunk UBA 5.4.2 Deployment Guide

This guide provides essential steps to deploy Splunk User Behavior Analytics (UBA) 5.4.2 on Red Hat Enterprise Linux (RHEL) 8.10. It includes hardware requirements, installation steps, SSL certificate management, data source configuration, and validation checklists for a successful deployment.

1. System Requirements

Hardware Requirements

- **CPU:** 16 CPU cores minimum per node
- **Memory:** 64 GB RAM minimum per node
- **Storage:**
 - **Disk 1:** 100 GB dedicated disk space for UBA installation
 - **Disk 2:** 1 TB additional disk space for metadata storage
 - **Disk 3:** 1 TB additional disk space for each node running Spark services
- **Network:** At least one 1 Gb ethernet interface per node
- **IOPS:** Disk subsystem must support average 1200 IOPS

Operating System Requirements

- **RHEL 8.10:** Target operating system for this deployment
- **Kernel Version:** 4.18.0-553.36.1.el8_10.x86_64 (tested)

⚠ **Important:** Installing Splunk UBA on hardened operating systems is not supported.

2. Networking Requirements

Static IP Address Configuration

All Splunk UBA servers must have static IP addresses assigned. DHCP is not supported.

Required Ports and Firewall Configuration

Configure firewall rules to allow the following ports:

External Access Ports

- **SSH:** 22 (for administrative access)
- **HTTPS:** 443 (for web interface access)

Internal Cluster Communication Ports

- **SSH:** 22
- **Redis:** 6379, 16379
- **PostgreSQL:** 5432
- **Zookeeper:** 2181, 2888, 3888
- **Apache Kafka:** 9092, 9901, 9093, 32768-65535
- **Job Manager:** 9002
- **Time Series Database (InfluxDB):** 8086
- **Apache Impala:** 21000, 21050, 25000, 25010, 25020
- **Apache Spark:** 7077, 8080, 8081
- **Hadoop HDFS:** 8020, 8090, 9866, 9867, 9864, 9868, 9870
- **Hive Metastore:** 9090, 9095

- **Kubernetes/etcd:** 2379, 2380, 5000, 6443, 10250, 10251, 10252, 10255, 30000-32767

Splunk Platform Communication Ports

- **HTTPS authentication:** 443
- **HTTP authentication:** 80
- **HTTP Event Collector:** 8088 (or 443 for Splunk Cloud)
- **REST services:** 8089
- **Splunk Universal Forwarder:** 9997

Outbound Ports

- **UBA telemetry:** 9997 (to *.splunkcloud.com)

DNS and Hostname Configuration

1. Configure hostname resolution:

```
# Ensure all nodes can resolve each other by hostname
host <hostname>
nslookup <hostname>

# Add entries to /etc/hosts if DNS is not available
echo "192.168.1.10 uba-node1.domain.com uba-node1" >> /etc/hosts
```

2. Verify network interface configuration:

```
# Check firewall zone assignment
sudo firewall-cmd --get-zone-of-interface=<interface_name>

# Verify default route is set
sudo ip route | grep "default"
```

3. Set proper network zone:

```
# Ensure the UBA interface is in the "public" zone
sudo firewall-cmd --zone=public --add-interface=<interface_name> --permanent
sudo firewall-cmd --reload
```

3. Pre-Installation Tasks

Configure Permissions and Prepare the User

1. Add the caspida user:

```
groupadd --gid 2018 caspida
useradd --uid 2018 --gid 2018 -m -d /home/caspida -c "Caspida User" -s /bin/bash caspida
passwd caspida
```

2. Enable sudo permissions:

```
# Edit the /etc/sudoers file using visudo
# Comment the line "Defaults requiretty" if it exists
# Add these lines at the end:
caspida ALL=(ALL) NOPASSWD:ALL
Defaults secure_path = /sbin:/bin:/usr/sbin:/usr/bin
```

3. Validate the UMASK value:

```
umask # Should return 0002 or 0022
grep -i "^UMASK" /etc/login.defs # Verify setting
```

Prepare Storage

1. Find 1TB disks:

```
fdisk -l
```

2. Partition and format disks (assuming /dev/sdb and /dev/sdc):

```
# For /dev/sdb
parted -a optimal /dev/sdb
mklabel gpt
mkpart primary ext4 2048s 100%
align-check opt 1
quit
mkfs -t ext4 /dev/sdb1

# For /dev/sdc
parted -a optimal /dev/sdc
mklabel gpt
mkpart primary ext4 2048s 100%
align-check opt 1
quit
mkfs -t ext4 /dev/sdc1
```

3. Get block IDs and mount disks:

```
blkid -o value -s UUID /dev/sdb1
blkid -o value -s UUID /dev/sdc1

# Create mount directories
mkdir -p /var/vcap /var/vcap2

# Add to /etc/fstab
# UUID=<sdb1-uuid> /var/vcap ext4 defaults 0 0
# UUID=<sdc1-uuid> /var/vcap2 ext4 defaults 0 0

# Mount the file systems
mount -a

# Set permissions
chmod 755 /var/vcap /var/vcap2
chown root:root /var/vcap /var/vcap2
```

Configure the Environment

1. Create caspida software directory:

```
mkdir /opt/caspida
chown caspida:caspida /opt/caspida
chmod 755 /opt/caspida
```

2. Set PostgreSQL environment variables:

```
# Add to /etc/locale.conf:
LANG="en_US.UTF-8"
LC_CTYPE="en_US.UTF-8"

# Apply changes
source /etc/locale.conf
```

3. Verify hostname resolution:

```
host <hostname>
```

4. Set SELinux to permissive mode:

```
# Edit /etc/sysconfig/selinux and set:
SELINUX=permissive
```

5. Configure system date, time and timezone:

```
timedatectl status
timedatectl set-timezone UTC # Set to match Splunk Enterprise
```

6. Configure kernel for bridge networking:

```
# Verify if bridge-nf-call-iptables exists and is set to 1
cat /proc/sys/net/bridge/bridge-nf-call-iptables

# If it doesn't exist:
modprobe br_netfilter
echo br_netfilter > /etc/modules-load.d/br_net_filter.conf
sysctl -w net.bridge.bridge-nf-call-iptables=1
echo net.bridge.bridge-nf-call-iptables=1 > /etc/sysctl.d/splunkuba-bridge.conf
```

4. Installation

Obtain and Prepare Installation Package

1. Download UBA software package:

```
# Download to /home/caspida directory:  
# splunk-uba-software-installation-package_542.tgz
```

2. Extract the package:

```
cd /home/caspida  
tar -xvzf splunk-uba-software-installation-package_542.tgz
```

Single Server Installation

1. Login as caspida user:

```
su - caspida
```

2. Extract the UBA platform files:

```
tar -xvf Splunk-UBA-Platform-5.4.2-*.tgz -C /opt/caspida
```

3. Run the installation script:

```
cd /opt/caspida/bin  
./INSTALL.sh -s
```

4. Follow the interactive prompts:

- Enter network interface (e.g., eth0)
- Enter IP address for Splunk UBA webUI
- Set admin password
- Confirm installation

Distributed Server Installation

1. Install on management node first:

```
cd /opt/caspida/bin  
./INSTALL.sh -c
```

2. During installation, provide:

- List of all node hostnames or IPs (comma-separated)
- Network interface name
- IP address for Splunk UBA webUI
- Admin password

3. Install on each remaining node:

```
cd /opt/caspida/bin  
./INSTALL.sh -n
```

5. Post-Installation Configuration

Verify Installation

1. Access the Splunk UBA web interface: https://<ip_address>
2. Log in with admin credentials
3. Check the health status:

```
/opt/caspida/bin/utils/uba_health_check.sh
```

Secure Default Account

1. Change the admin password:

```
# Log in to Splunk UBA UI  
# Go to Manage > UBA Accounts  
# Edit the admin user and change password
```

Basic Service Management

- Start all services:

```
/opt/caspida/bin/Caspida start-all
```

- Stop all services:

```
/opt/caspida/bin/Caspida stop-all
```

- Restart UI:

```
sudo service caspida-ui restart
```

6. Configure Splunk UBA

Upload Splunk UBA License

After installation, you must upload a valid license file:

1. Access the Splunk UBA web interface
2. Navigate to **Manage > License**
3. Click **Choose File** and select your license file
4. Click **Open** to upload

Important: Without a valid license, UBA will not ingest data. Data sources will show as “Suspended.”

(Optional) Enable FIPS Compliance

Federal Information Processing Standard (FIPS) compliance is available for UBA 5.4.0 and higher.

For RHEL 8.10 Systems:

1. Check current FIPS status:

```
sudo fips-mode-setup --check
```

2. Enable FIPS mode:

```
sudo fips-mode-setup --enable
```

3. Reboot the system:

```
sudo reboot
```

4. Verify FIPS is enabled:

```
sudo fips-mode-setup --check  
cat /proc/sys/crypto/fips_enabled # Should return 1
```

Note: Enable FIPS on each UBA node before running installation/upgrade scripts.

SSL Certificate Management

Updating the Default Self-Signed Certificate

The self-signed certificate included with Splunk UBA expires 365 days after the Splunk UBA web interface is accessed for the first time.

1. Regenerate the default self-signed certificate:

```
rm /var/vcap/store/caspida/certs/my-root-ca.crt.pem  
/opt/caspida/bin/CaspidaCert.sh US CA "San Francisco" Splunk "" "" /var/vcap/store/caspida/certs/
```

Creating and Installing Third-Party Certificates

For production environments, you should replace the self-signed certificate with a third-party signed certificate.

Generate a Certificate Signing Request (CSR)

1. Stop the Splunk UBA services:

```
sudo service caspida-resourcesmonitor stop  
sudo service caspida-ui stop
```

2. Get host information:

```
hostname -s # Short hostname
hostname -d # Domain name
```

3. Generate certificates:

```
sudo /opt/caspida/bin/CaspidaCert.sh <country> <state> <location> <org> <domain> <"short hostname"> /var/vcap/store/caspida/certs/mycerts
```

Example:

```
sudo /opt/caspida/bin/CaspidaCert.sh US CA SanFrancisco Splunk sv.splunk.com "uba-17" /var/vcap/store/caspida/certs/mycerts
```

4. Update uba-site.properties:

```
# Add the following to /etc/caspida/local/conf/uba-site.properties
ui.auth.rootca=/var/vcap/store/caspida/certs/mycerts/my-root-ca.crt.pem
ui.auth.privateKey=/var/vcap/store/caspida/certs/mycerts/my-server.key.pem
ui.auth.serverCert=/var/vcap/store/caspida/certs/mycerts/my-server.crt.pem
```

5. Generate CSR for the certificate authority:

```
cd /var/vcap/store/caspida/certs/mycerts
sudo openssl req -new -key my-server.key.pem -out myCACertificate.csr
```

6. Set permissions:

```
sudo chmod 644 /var/vcap/store/caspida/certs/mycerts/*
```

7. Start services:

```
sudo service caspida-ui start
sudo service caspida-resourcesmonitor start
```

8. Submit the CSR to your certificate authority and obtain signed certificates.

Installing Third-Party Signed Certificates

1. Stop services:

```
sudo service caspida-resourcesmonitor stop
sudo service caspida-ui stop
```

2. Back up existing certificates:

```
sudo cp -p /var/vcap/store/caspida/certs/my-server.crt.pem /var/vcap/store/caspida/certs/my-server.crt.pem_backup
sudo cp -p /var/vcap/store/caspida/certs/my-root-ca.crt.pem /var/vcap/store/caspida/certs/my-root-ca.crt.pem_backup
```

3. Install new certificates:

```
sudo mv -f /path/to/your/signed/certificate.pem /var/vcap/store/caspida/certs/mycerts/my-server.crt.pem
sudo mv -f /path/to/your/root/certificate.pem /var/vcap/store/caspida/certs/mycerts/my-root-ca.crt.pem
```

4. Set permissions:

```
sudo chmod 644 /var/vcap/store/caspida/certs/mycerts/*
```

5. For distributed deployments, synchronize configuration:

```
/opt/caspida/bin/Caspida sync-cluster /etc/caspida/local/conf
```

6. Start services:

```
sudo service caspida-ui start
sudo service caspida-resourcesmonitor start
```

7. Verify certificate installation by accessing the UBA web interface.

Handling Subject Alternative Names (SAN)

If Chrome reports ERR_CERT_COMMON_NAME_INVALID, you need to include Subject Alternative Names:

1. Create directory and configuration:

```
mkdir -p /opt/caspida/conf/deployment/templates/local_conf/ssl
```

2. Create openssl-altname.cnf:

```
cat > /opt/caspida/conf/deployment/templates/local_conf/ssl/openssl-altname.cnf << 'EOF'
[ req ]
default_bits       = 2048
distinguished_name = req_distinguished_name
attributes         = req_attributes
[ req_distinguished_name ]
countryName               = Country Name (2 letter code)
stateOrProvinceName       = State or Province Name (full name)
localityName              = Locality Name (eg, city)
organizationName          = Organization Name (eg, company)
organizationalUnitName     = Organizational Unit Name (eg, section)
commonName                = Common Name (e.g. server FQDN or YOUR name)
[req_attributes]
subjectAltName            = Alternative DNS names, Email addresses or IPs (comma separated)
EOF
```

3. Generate CSR with SANs:

```
openssl req -sha256 -new -key my-server.key.pem -out myCACertificate.csr -config /opt/caspida/conf/deployment/templates/local_conf/ssl/openssl-
altname.cnf
```

When prompted for alternative names, enter: **DNS:hostname.domain.com, DNS:hostname, IP:192.168.0.1**

4. Verify SANs in the CSR:

```
openssl req -text -noout -verify -in myCACertificate.csr | grep DNS
```

Replace Job Manager Certificate

For communication between UBA components:

1. Stop services:

```
/opt/caspida/bin/Caspida stop
```

2. Generate the keystore:

```
cd /opt/caspida/bin/jobmanager
sudo ./generate-keystore.sh
```

3. Start services:

```
/opt/caspida/bin/Caspida start
```

7. Manage User Accounts and Authentication

User Account Roles

UBA supports multiple user roles with different privilege levels:

- **Admin (uba_admin):** Full system administration access
- **Analyst (uba_analyst):** Security analysis capabilities
- **Content Developer (uba_content_developer):** Custom content creation
- **PII Unmask (uba_pii_unmask):** Access to unmasked PII data
- **User (uba_user):** Basic viewing permissions

Configure Single Sign-On (SSO)

For enterprise authentication integration:

1. Prepare SSO configuration:

- Obtain identity provider metadata file
- Ensure DNS resolution for identity provider
- Configure role mappings

2. Configure SSO in UBA:

```
# Navigate to Manage > Settings > Authentication
# Select "Single Sign-On" authentication type
# Upload identity provider metadata file
# Configure role mappings
```


3. Supported identity providers:

- Microsoft Active Directory Federation Services (ADFS)
- Microsoft Entra ID (Azure AD)
- OneLogin
- Generic SAML 2.0 providers

4. Test SSO configuration:

- Verify role assignments
- Test user login flow
- Validate group mappings

Configure Splunk Platform Authentication

For Splunk Enterprise/Cloud integration:

1. Create service account in Splunk:

```
# On Splunk Enterprise
# Navigate to Settings > Access controls > Users
# Create user with appropriate roles
```

2. Configure in UBA:

```
# Navigate to Data Sources > Add Data Source
# Select "Splunk Data"
# Enter Splunk credentials
# Test connection
```

8. Backup and Restore Configuration

Configure Automated Incremental Backups

For disaster recovery and high availability:

1. Set backup properties in uba-site.properties:

```
# Edit /etc/caspida/local/conf/uba-site.properties
backup.filesystem.enabled=true
backup.filesystem.directory=/backup
backup.filesystem.full.interval=1 week
```

2. Synchronize changes in distributed deployment:

```
/opt/caspida/bin/Caspida sync-cluster /etc/caspida/local/conf
```

3. Create and configure backup directory:

```
sudo mkdir -p /backup
sudo chown caspida:caspida /backup
sudo chmod 755 /backup
```

4. Verify backup configuration:

```
grep backup.filesystem /etc/caspida/local/conf/uba-site.properties
```

Manual Full Backup Procedure

For major upgrades or migrations:

1. Stop all services:

```
/opt/caspida/bin/Caspida stop-all
```

2. Run backup script:

```
/opt/caspida/bin/Utils/backup.sh -d /backup -a
```

Options:

- **-d /backup**: Backup directory path
- **-a**: Create archive
- **-c**: Include container data

- **-s**: Skip regular backups for selective backup

3. Start all services after backup:

```
/opt/caspida/bin/Caspida start-all
```

4. Verify backup was successful:

```
ls -la /backup/
```

Restore Procedure

In case of system failure or migration:

1. Stop all services:

```
/opt/caspida/bin/Caspida stop-all
```

2. Run restore script:

```
/opt/caspida/bin/Utils/restore.sh -d /backup/<backup-timestamp> -a
```

Options:

- **-d /backup/<backup-timestamp>**: Path to backup directory
- **-a**: Restore from archive
- **-c**: Include container data
- **-s**: Selective restore

3. Start all services after restore:

```
/opt/caspida/bin/Caspida start-all
```

9. Configure Warm Standby for High Availability

For critical deployments requiring high availability:

1. Configure warm standby properties:

```
# On primary system, edit /etc/caspida/local/conf/uba-site.properties
replication.enabled=true
replication.primary.host=<primary-hostname>
replication.standby.host=<standby-hostname>
```

2. Setup replication on primary node:

```
/opt/caspida/bin/replication/setup standby -m primary -r
```

3. Setup replication on standby node:

```
/opt/caspida/bin/replication/setup standby -m standby -r
```

4. Initiate full synchronization:

```
curl -X POST -k -H "Authorization: Bearer $(grep '^s*jobmanager.restServer.auth.user.token=' /opt/caspida/conf/uba-default.properties | cut -d
f2)" https://localhost:9002/jobs/trigger?name=ReplicationCoordinator
```

5. Verify synchronization status:

```
psql -d caspidadb -c 'select * from replication'
```

10. Monitor Splunk UBA

Health Monitoring

Regular health monitoring is critical for maintaining optimal performance:

1. **Run health check script:**

```
/opt/caspida/bin/Utils/uba_health_check.sh
```

2. Access Health Monitor dashboard:

```
# Navigate to System > Health Monitor in UBA UI
```

3. Monitor key metrics:

- System resource utilization (CPU, memory, disk)
- Service status and uptime
- Data ingestion rates (EPS)
- Queue depths and processing delays
- Certificate expiration dates

4. Set up automated monitoring:

```
# Configure resource monitor
sudo service caspida-resourcesmonitor start

# Monitor logs
tail -f /var/log/caspida/monitor/resourcesMonitor.out
```

Performance Optimization

Optimize UBA performance based on workload:

1. Adjust data ingestion parameters:

```
# Edit /etc/caspida/local/conf/uba-site.properties
splunk.live.micro.batching.interval.seconds=60
splunk.live.micro.batching.delay.seconds=180
connector.splunk.max.backtrace.time.in.hour=4
```

2. Configure anomaly retention:

```
# Edit /etc/caspida/local/conf/uba-site.properties
persistence.anomalies.trashed.maintain.days=90
entity.score.lookbackWindowMonths=2
```

3. Optimize threat rule processing:

```
# Edit /etc/caspida/local/conf/uba-site.properties
rule.engine.process.timeout.min=60
```

4. Monitor and adjust container resources:

```
# Check container resource usage
docker stats

# Adjust Docker network if needed
system.docker.networkcidr=172.18.0.0/16
```

Collect Diagnostic Data

For troubleshooting and support:

1. Generate diagnostic bundle:

```
# Navigate to System > Download Diagnostics
# Select components to include
# Download generated bundle
```

2. Manual log collection:

```
# Key log locations
/var/vcap/sys/log/caspida-ui/
/var/vcap/sys/log/caspida-jobmanager/
/var/vcap/sys/log/caspida-datasource/
/var/log/caspida/
```

11. Customize Splunk UBA

Configure Internal IP Ranges

For proper classification of internal vs external traffic:

1. Edit the EntityValidations.json file:

```
cd /etc/caspida/local/conf/etl/configuration
vi EntityValidations.json
```

2. Add your internal IP ranges in CIDR notation:

```
"internalIPRange": ["199.79.0.0/16", "220.200.0.0/16"]
```

3. Optionally add office location mapping:

```
"internalGeoAttributions": [
  {"cidr": "10.0.0.0/8", "location": {"city": "San Jose", "countryCode": "US", "latitude": 37.3382, "longitude": -121.8863}},
  {"cidr": "172.16.0.0/16", "location": {"city": "New York", "countryCode": "US", "latitude": 40.788614, "longitude": -73.9696091}}
]
```

4. Validate the configuration:

```
/opt/caspida/bin/status/check_entity_validations.sh -v
```

5. Synchronize changes across nodes:

```
/opt/caspida/bin/Caspida sync-cluster /etc/caspida/local/conf/etl/configuration
```

Geo-Location Settings

For accurate location-based analytics:

1. Navigate to Manage > Settings > Geo Location
2. Configure default office location with:
 - Latitude and longitude coordinates
 - Location name
 - Enable “Show Geo Maps” option

Email Alerts Configuration

For system alerts and notifications:

1. First set up email output connector:

```
# Navigate to Manage > Output Connectors > Add Output Connector > Email
```

2. Configure system alerts:

- Navigate to Manage > Settings > Alerts
- Add administrator email addresses (one per line)
- Enable “Alert when processing stops” option
- Set minimum EPS threshold
- Set alert interval in seconds

Customize Anomaly and Threat Parameters

1. Set entity score lookback window:

```
# Edit /etc/caspida/local/conf/uba-site.properties
entity.score.lookbackWindowMonths=2
```

2. Configure anomaly retention period:

```
# Edit /etc/caspida/local/conf/uba-site.properties
persistence.anomalies.trashed.maintain.days=90
```

3. Set anomaly purge batch size:

```
# Edit /etc/caspida/local/conf/uba-site.properties
persistence.anomalies.trashed.del.limit=300000
```

4. Configure threat rule engine timeout:

```
# Edit /etc/caspida/local/conf/uba-site.properties
rule.engine.process.timeout.min=60
```

5. Synchronize and restart required services:

```
/opt/caspida/bin/Caspida sync-cluster /etc/caspida/local/conf
/opt/caspida/bin/Caspida stop-containers
/opt/caspida/bin/Caspida start-containers
```

Masking PII Information

For compliance with privacy regulations:

1. Navigate to Manage > Settings > PII
2. Configure masking for sensitive fields:
 - User information
 - Threat information
 - Device information
3. Set masking level:
 - Full masking: Complete obfuscation
 - Partial masking: Show limited identifiable information

Disable UI Timeout (Optional)

For installations that need continuous monitoring:

1. Set timeout property (in milliseconds):

```
# Edit /etc/caspida/local/conf/uba-site.properties
# To disable timeout:
ui.idleTimeout=0
# For 1-hour timeout:
ui.idleTimeout=3600000
```

2. Restart UI service:

```
sudo service caspida-ui restart
```

12. Data Source Configuration

Required Data Sources

For effective threat detection, the following data sources should be configured in priority order:

1. HR Data Source

This is critical for user contextual information and proper identity resolution.

1. Configure HR data source:

```
# Navigate to Data Sources > Add Data Source > HR Data Source
```

2. Required fields mapping:

- User ID (required)
- Full Name
- Email Address
- Manager ID (for reporting hierarchy)
- Title/Role
- Department
- Location
- Start Date
- Termination Date

3. Configure refresh schedule:

- By default, HR data is refreshed daily at 2:00 AM

2. Assets Data Source

For proper device-to-user mapping and network asset inventory.

1. Configure assets data source:

```
# Navigate to Data Sources > Add Data Source > Assets Data Source
```

2. Required fields mapping:

- Asset ID (hostname or MAC)
- IP Address
- Owner (User ID)
- Asset Type
- Location (optional)

3. Configure multi-value fields if needed:

```
# Set delimiter in uba-site.properties
attribution.keyvalue.delimiter=,
```

3. Authentication Data Source

For detecting credential abuse and suspicious logins.

1. Configure authentication data source:

```
# Navigate to Data Sources > Add Data Source > Authentication Data
# Select appropriate source type (Active Directory, VPN, etc.)
```

2. Required fields mapping:

- User ID
- Success/Failure status
- Timestamp
- Source IP
- Event Type (login, logout, etc.)

4. Network/Proxy Data

For detecting unusual communication patterns or data exfiltration.

1. Configure proxy data source:

```
# Navigate to Data Sources > Add Data Source > Proxy/Web Data
```

2. Field mapping:

- User ID
- Source IP
- Destination IP/URL
- Timestamp
- Bytes Transferred

Splunk Data Source Configuration

For ingesting data from Splunk Enterprise or Splunk Cloud:

1. Add Splunk data source:

```
# Navigate to Data Sources > Add Data Source > Splunk Data
```

2. Enter connection details:

- Splunk Host
- Splunk Management Port
- Username
- Password
- Connection Protocol (HTTP/HTTPS)

3. Select data ingestion method:

- Micro-batching (default) - Recommended for most deployments
- Configure in uba-site.properties:

```
splunk.live.micro.batching=true
splunk.live.micro.batching.delay.seconds=180
splunk.live.micro.batching.interval.seconds=60
connector.splunk.max.backtrace.time.in.hour=4
```

SSL Certificate Validation for Data Sources

For secure data source connections:

1. Enable SSL certificate validation:

```
# In uba-site.properties
validate.splunk.ssl.certificate=true
```

2. Import SSL certificates for HTTPS data sources:

```
# Copy Splunk certificate
scp user@splunk-host:/opt/splunk/etc/auth/ca.pem.default /home/caspida/

# Import certificate to Java keystore
. /opt/caspida/bin/CaspidaCommonEnv.sh
sudo keytool -keystore $JAVA_HOME/lib/security/cacerts -storepass changeit -import -trustcacerts -alias SplunkESRootCA -file ~/ca.pem.default
```

3. Restart services:

```
/opt/caspida/bin/Caspida stop-all
/opt/caspida/bin/Caspida start-all
```

Validate Data Ingestion

1. Check data source status:

```
# Navigate to Data Sources > [Data Source Name]
```

2. Verify EPS (Events Per Second) is non-zero
3. Check event processing in UBA:

```
# Check processing logs
tail -f /var/vcap/sys/log/caspida-datasource/caspida-datasource.stderr.log
```

4. Validate identity resolution:

```
# Navigate to Manage > IDR Exclusions
# Ensure user-device associations are properly established
```

13. Send Data from Splunk UBA

Send UBA Data to Splunk Enterprise Security

For integration with Splunk ES:

1. **Configure ES integration:**

```
# Edit /etc/caspida/local/conf/uba-site.properties
uba.splunkes.retry.delay.minutes=5
uba.sys.audit.push.splunk.enabled=true
identity.resolution.export.enabled=true
```

2. **Configure threat synchronization:**

- UBA automatically sends threats to Splunk ES
- Threats appear as notables in ES
- Configure sync frequency as needed

3. **Send user and device associations:**

- Enable identity resolution export
- Data appears in Session Center dashboard in ES

Send Threats via Email

Configure email notifications for threats:

1. **Set up email output connector:**

```
# Navigate to Manage > Output Connectors > Add Output Connector > Email
```

2. **Configure SMTP settings:**

- SMTP server hostname
- Port (typically 25, 587, or 465)

- Authentication credentials
- SSL/TLS settings

3. Create threat notification rules:

- Configure which threats trigger emails
- Set recipient lists
- Customize email templates

Send Threats to ServiceNow

For ITSM integration:

1. Configure ServiceNow connector:

```
# Navigate to Manage > Output Connectors > Add Output Connector > ServiceNow
```

2. Provide ServiceNow details:

- ServiceNow instance URL
- Username and password
- Table name (typically incident)

3. Configure threat mapping:

- Map UBA threat fields to ServiceNow fields
- Set priority and category mappings

14. Security Hardening and Best Practices

Secure Default Account

After installation, immediately secure the default admin account:

1. Change default admin password:

```
# Navigate to Manage > UBA Accounts
# Edit admin user
# Set strong password
```

2. Restrict sudo access (optional):

```
# Edit /etc/sudoers to limit caspida user permissions
# Remove NOPASSWD:ALL if desired
# Add specific command permissions only
```

Third-Party Security Agents

Install security agents only AFTER UBA installation:

1. Exclude UBA directories from scanning:

```
# Add these paths to security agent exclusions:
/var/vcap/store/docker/overlay2/
/opt/caspida/
/var/vcap/
```

2. Monitor resource impact:

```
# Check if security agents cause high CPU usage
top
htop
```

Regular Security Updates

Apply OS security patches regularly on RHEL 8.10:


```
# Stop UBA services
/opt/caspida/bin/Caspida stop-all

# Check for security updates
sudo yum updateinfo list security all

# Apply security updates
sudo yum update --security -y
sudo yum --security update-minimal

# Reboot if required
sudo reboot

# Start UBA services
/opt/caspida/bin/Caspida start-all
```

Certificate Security

Maintain certificate security:

1. Monitor certificate expiration:

```
# Check certificate validity
openssl x509 -in /var/vcap/store/caspida/certs/my-server.crt.pem -text -noout | grep -E 'Not Before|Not After'
```

2. Set up certificate renewal alerts:

- Monitor certificates that expire within 30 days
- Configure automated renewal if possible
- Plan certificate replacement procedures

3. Secure certificate storage:

```
# Ensure proper permissions
chmod 644 /var/vcap/store/caspida/certs/*/*
chown root:root /var/vcap/store/caspida/certs/*/*
```

15. Troubleshooting

For comprehensive troubleshooting procedures covering installation issues, service problems, data source issues, SSL certificate problems, performance issues, and emergency recovery procedures, refer to the separate **Splunk_UBA_Troubleshooting_Guide.md** document.

Quick Reference - Common Commands

```
# Check service status
/opt/caspida/bin/Caspida status

# Run health check
/opt/caspida/bin/Utils/uba_health_check.sh

# Emergency service restart
/opt/caspida/bin/Caspida stop-all
/opt/caspida/bin/Caspida start-all

# Reset admin password
/opt/caspida/bin/admin_password_reset.sh
```

Common Log Locations

- UI: **/var/vcap/sys/log/caspida-ui/caspida-ui.stderr.log**
- Job Manager: **/var/vcap/sys/log/caspida-jobmanager/caspida-jobmanager.stderr.log**
- Data Sources: **/var/vcap/sys/log/caspida-datasource/caspida-datasource.stderr.log**
- System: **/var/log/caspida/caspida.out**

16. Advanced Administration

Managing UBA Configuration Properties

Configure UBA by editing **/etc/caspida/local/conf/uba-site.properties** :

1. Environment Properties:

```
# Docker network CIDR (avoid network conflicts)
system.docker.networkcidr=172.18.0.0/16

# UI idle timeout (30 minutes default)
ui.idleTimeout=1800000
```

2. Splunk ES Integration:

```
# Threat sync frequency
uba.splunkes.retry.delay.minutes=5

# Enable audit events to Splunk ES
uba.sys.audit.push.splunk.enabled=true

# Send user/device associations to ES
identity.resolution.export.enabled=true
```

3. Event Drilldown Properties:

```
# Anomaly threshold for caching SPL
triggering.event.pre.calculate.links.anomaly.threshold=8

# Timeout for SPL retrieval
triggering.event.timeout.millis=300000

# Enable reverse identity resolution
triggering.event.enable.reverse.ir=false
```

4. Data Ingestion Properties:

```
# Micro-batching settings
splunk.live.micro.batching=true
splunk.live.micro.batching.delay.seconds=180
splunk.live.micro.batching.interval.seconds=60

# Backfill window
connector.splunk.max.backtrace.time.in.hour=4

# Input timezone for file-based sources
parser.global.input_timezone=UTC
```

Start and Stop Services

Common service management commands:

1. Service Control:

```
# Stop/start all UBA services
/opt/caspida/bin/Caspida stop
/opt/caspida/bin/Caspida start

# Stop/start all services including platform services
/opt/caspida/bin/Caspida stop-all
/opt/caspida/bin/Caspida start-all

# Individual service management
sudo service caspida-ui restart
sudo service caspida-resourcesmonitor restart
```

2. Container Management:

```
# Stop/start containers only
/opt/caspida/bin/Caspida stop-containers
/opt/caspida/bin/Caspida start-containers

# Data source management
/opt/caspida/bin/Caspida stop-datasources
/opt/caspida/bin/Caspida start-datasources
```

3. Configuration Synchronization:

```
# Sync configuration changes across cluster
/opt/caspida/bin/Caspida sync-cluster /etc/caspida/local/conf
```

Change Service Passwords

For changing passwords of UBA services:

1. Generate new password:

```
/opt/caspida/bin/password_manager.sh generate
```

2. Update specific service password:

```
/opt/caspida/bin/password_manager.sh update <service_name>
```

3. Restart affected services:

```
/opt/caspida/bin/Caspida restart
```

Change IP Address or Hostname

If you need to change the IP address or hostname:

1. **Update network configuration on OS level**
2. **Update UBA configuration:**

```
# Edit uba-site.properties
uiServer.host=<new-hostname-or-ip>
```

3. **Update deployment configuration:**

```
# Edit deployment configuration
vi /opt/caspida/conf/deployment/caspida-deployment.conf
```

4. **Synchronize and restart:**

```
/opt/caspida/bin/Caspida sync-cluster /etc/caspida/local/conf
/opt/caspida/bin/Caspida restart
```

17. Service Management Migration and Non-Privileged Operations

Switching from Script-Managed to systemctl-Managed Services

UBA can be managed either through traditional Caspida scripts (`/opt/caspida/bin/Caspida`) or modern systemd services. This section covers migrating to systemctl management for better system integration.

Current Service Management Assessment

1. **Check current service management method:**

```
# Check if systemd services exist
systemctl list-units --type=service | grep caspida

# Check running UBA processes
ps aux | grep -E "(caspida|java.*uba)" | grep -v grep

# Test script-based management
/opt/caspida/bin/Caspida status
```

2. **Determine migration readiness:**

```
# Verify UBA installation completeness
ls -la /opt/caspida/bin/
ls -la /etc/caspida/

# Check systemd capability
systemctl --version
systemctl status | head -5
```

Creating systemd Service Units

1. **Create systemd service files for core UBA services:**

```

# Create UI service unit
sudo tee /etc/systemd/system/caspida-ui.service > /dev/null << 'EOF'
[Unit]
Description=Splunk UBA UI Service
After=network.target
Wants=network.target

[Service]
Type=forking
User=caspida
Group=caspida
Environment="JAVA_HOME=/opt/caspida/third-party/java/current"
Environment="CASPIDA_HOME=/opt/caspida"
ExecStart=/opt/caspida/bin/containers/caspida-ui start
ExecStop=/opt/caspida/bin/containers/caspida-ui stop
ExecReload=/opt/caspida/bin/containers/caspida-ui restart
PIDFile=/var/vcap/sys/run/caspida-ui/caspida-ui.pid
TimeoutStartSec=300
TimeoutStopSec=300
Restart=no

[Install]
WantedBy=multi-user.target
EOF

# Create Job Manager service unit
sudo tee /etc/systemd/system/caspida-jobmanager.service > /dev/null << 'EOF'
[Unit]
Description=Splunk UBA Job Manager Service
After=network.target postgresql.service
Wants=network.target
Requires=postgresql.service

[Service]
Type=forking
User=caspida
Group=caspida
Environment="JAVA_HOME=/opt/caspida/third-party/java/current"
Environment="CASPIDA_HOME=/opt/caspida"
ExecStart=/opt/caspida/bin/containers/caspida-jobmanager start
ExecStop=/opt/caspida/bin/containers/caspida-jobmanager stop
ExecReload=/opt/caspida/bin/containers/caspida-jobmanager restart
PIDFile=/var/vcap/sys/run/caspida-jobmanager/caspida-jobmanager.pid
TimeoutStartSec=300
TimeoutStopSec=300
Restart=no

[Install]
WantedBy=multi-user.target
EOF

# Create Resource Monitor service unit
sudo tee /etc/systemd/system/caspida-resourcesmonitor.service > /dev/null << 'EOF'
[Unit]
Description=Splunk UBA Resource Monitor Service
After=network.target
Wants=network.target

[Service]
Type=forking
User=caspida
Group=caspida
Environment="JAVA_HOME=/opt/caspida/third-party/java/current"
Environment="CASPIDA_HOME=/opt/caspida"
ExecStart=/opt/caspida/bin/containers/caspida-resourcesmonitor start
ExecStop=/opt/caspida/bin/containers/caspida-resourcesmonitor stop
ExecReload=/opt/caspida/bin/containers/caspida-resourcesmonitor restart
PIDFile=/var/vcap/sys/run/caspida-resourcesmonitor/caspida-resourcesmonitor.pid
TimeoutStartSec=300
TimeoutStopSec=300
Restart=no

[Install]
WantedBy=multi-user.target
EOF

```

2. Reload systemd and enable services:

```

# Reload systemd configuration
sudo systemctl daemon-reload

# Verify service files are valid
sudo systemctl status caspida-ui.service
sudo systemctl status caspida-jobmanager.service
sudo systemctl status caspida-resourcesmonitor.service

# Enable services for automatic startup
sudo systemctl enable caspida-ui.service
sudo systemctl enable caspida-jobmanager.service
sudo systemctl enable caspida-resourcesmonitor.service

```

Migration Procedure

1. Stop services using existing method:

```
# Stop all UBA services cleanly
/opt/caspida/bin/Caspida stop-all

# Verify all services are stopped
ps aux | grep -E "(caspida|java.*uba)" | grep -v grep
sleep 30
```

2. Test systemd service startup:

```
# Start services individually to test
sudo systemctl start caspida-jobmanager.service
sleep 60
sudo systemctl status caspida-jobmanager.service

sudo systemctl start caspida-ui.service
sleep 30
sudo systemctl status caspida-ui.service

sudo systemctl start caspida-resourcesmonitor.service
sleep 30
sudo systemctl status caspida-resourcesmonitor.service
```

3. Verify functionality:

```
# Check UBA health
/opt/caspida/bin/Utils/uba_health_check.sh

# Test UI accessibility
curl -k -I https://localhost/

# Test Job Manager API
curl -k -I https://localhost:9002/health
```

4. Create service management wrapper script:

```

# Create hybrid management script
sudo tee /usr/local/bin/caspida-service-manager > /dev/null << 'EOF'
#!/bin/bash
# Caspida Service Management Wrapper
# Supports both systemd and traditional script management

SERVICES=("caspida-jobmanager" "caspida-ui" "caspida-resourcesmonitor")

function check_systemd() {
    systemctl list-units --type=service | grep -q caspida
}

function start_all() {
    if check_systemd; then
        echo "Starting UBA services via systemd..."
        for service in "${SERVICES[@]"; do
            sudo systemctl start "$service"
            echo "Started $service"
        done
    else
        echo "Starting UBA services via scripts..."
        /opt/caspida/bin/Caspida start-all
    fi
}

function stop_all() {
    if check_systemd; then
        echo "Stopping UBA services via systemd..."
        for service in "${SERVICES[@]"; do
            sudo systemctl stop "$service"
            echo "Stopped $service"
        done
    else
        echo "Stopping UBA services via scripts..."
        /opt/caspida/bin/Caspida stop-all
    fi
}

function status_all() {
    if check_systemd; then
        echo "UBA service status via systemd:"
        for service in "${SERVICES[@]"; do
            systemctl is-active "$service" || echo "$service: inactive"
        done
    else
        echo "UBA service status via scripts:"
        /opt/caspida/bin/Caspida status
    fi
}

case "$1" in
    start)
        start_all
        ;;
    stop)
        stop_all
        ;;
    status)
        status_all
        ;;
    restart)
        stop_all
        sleep 30
        start_all
        ;;
    *)
        echo "Usage: $0 {start|stop|restart|status}"
        exit 1
        ;;
esac
EOF

# Make executable
sudo chmod +x /usr/local/bin/caspida-service-manager

```

Configuring Non-Privileged UBA Instance

For enhanced security, UBA can be configured to run with minimal privileges and restricted sudo access.

Security Assessment and Planning

1. Current privilege assessment:

```

# Check current caspida user privileges
sudo -l -U caspida

# Review current sudo configuration
sudo grep -r caspida /etc/sudoers*

# Check file ownership
find /opt/caspida -user root -ls | head -10
find /var/vcap -user root -ls | head -10

```

2. Identify required privileges:

```
# Services that need specific access
echo "Required UBA service privileges:"
echo "- Network port binding (>1024 for non-privileged)"
echo "- File system access (/opt/caspida, /var/vcap, /etc/caspida)"
echo "- Log file management"
echo "- Process management (start/stop own services)"
echo "- Certificate management (if SSL enabled)"
```

Implementing Non-Privileged Configuration

1. Create restricted sudo configuration:

```
# Create specific sudoers file for caspida
sudo tee /etc/sudoers.d/10-caspida-restricted > /dev/null << 'EOF'
# Restricted sudo access for Caspida user
# Allow only specific service management commands

Defaults:caspida !requiretty, env_keep += "JAVA_HOME CASPIDA_HOME"

# Service management - only for caspida services
caspida ALL=(ALL) NOPASSWD: /bin/systemctl start caspida-*
caspida ALL=(ALL) NOPASSWD: /bin/systemctl stop caspida-*
caspida ALL=(ALL) NOPASSWD: /bin/systemctl restart caspida-*
caspida ALL=(ALL) NOPASSWD: /bin/systemctl status caspida-*
caspida ALL=(ALL) NOPASSWD: /bin/systemctl reload caspida-*

# Platform services management (if co-located)
caspida ALL=(ALL) NOPASSWD: /bin/systemctl start postgresql
caspida ALL=(ALL) NOPASSWD: /bin/systemctl stop postgresql
caspida ALL=(ALL) NOPASSWD: /bin/systemctl restart postgresql
caspida ALL=(ALL) NOPASSWD: /bin/systemctl status postgresql

caspida ALL=(ALL) NOPASSWD: /bin/systemctl start kafka-server
caspida ALL=(ALL) NOPASSWD: /bin/systemctl stop kafka-server
caspida ALL=(ALL) NOPASSWD: /bin/systemctl restart kafka-server
caspida ALL=(ALL) NOPASSWD: /bin/systemctl status kafka-server

caspida ALL=(ALL) NOPASSWD: /bin/systemctl start zookeeper-server
caspida ALL=(ALL) NOPASSWD: /bin/systemctl stop zookeeper-server
caspida ALL=(ALL) NOPASSWD: /bin/systemctl restart zookeeper-server
caspida ALL=(ALL) NOPASSWD: /bin/systemctl status zookeeper-server

# Log file access
caspida ALL=(ALL) NOPASSWD: /bin/tail /var/log/caspida/*
caspida ALL=(ALL) NOPASSWD: /bin/cat /var/log/caspida/*
caspida ALL=(ALL) NOPASSWD: /usr/bin/less /var/log/caspida/*

# Certificate management (if needed)
caspida ALL=(ALL) NOPASSWD: /usr/bin/keytool -list -keystore *
caspida ALL=(ALL) NOPASSWD: /usr/bin/openssl verify *
caspida ALL=(ALL) NOPASSWD: /usr/bin/openssl x509 *

# Disk space and system monitoring
caspida ALL=(ALL) NOPASSWD: /bin/df -h
caspida ALL=(ALL) NOPASSWD: /usr/bin/du -sh /opt/caspida*
caspida ALL=(ALL) NOPASSWD: /usr/bin/du -sh /var/vcap*
caspida ALL=(ALL) NOPASSWD: /bin/free -m

# Network diagnostics
caspida ALL=(ALL) NOPASSWD: /bin/netstat -tulpn
caspida ALL=(ALL) NOPASSWD: /usr/sbin/ss -tulpn
caspida ALL=(ALL) NOPASSWD: /usr/sbin/lsof -i *

# Process management for owned processes only
caspida ALL=(caspida) NOPASSWD: /bin/kill -TERM [0-9]*
caspida ALL=(caspida) NOPASSWD: /bin/kill -HUP [0-9]*
caspida ALL=(caspida) NOPASSWD: /usr/bin/pkill -f caspida*

# Deny all other sudo access
caspida ALL=(ALL) !/bin/su*, !/bin/bash, !/bin/sh, !/usr/bin/vi*, !/usr/bin/nano, !/usr/bin/emacs
caspida ALL=(ALL) !/usr/bin/passwd*, !/usr/sbin/user*, !/usr/sbin/group*
caspida ALL=(ALL) !/bin/chmod [0-7][0-7][0-7]*, !/bin/chown*
caspida ALL=(ALL) !/usr/bin/rpm*, !/usr/bin/yum*, !/usr/bin/dnf*
EOF

# Validate sudoers syntax
sudo visudo -c -f /etc/sudoers.d/10-caspida-restricted
```

2. Configure non-privileged port binding:

```
# Configure UBA to use non-privileged ports
sudo -u caspida tee -a /etc/caspida/local/conf/uba-site.properties > /dev/null << 'EOF'

# Non-privileged port configuration
# UI Server (use port 8080 instead of 80, 8443 instead of 443)
uiServer.port=8080
uiServer.ssl.port=8443

# Job Manager (default 9002 is already >1024)
jobmanager.restServer.port=9002

# Resource Monitor
resourcesmonitor.port=9003
EOF
```

3. Update systemd services for non-privileged operation:

```
# Update service files to include security restrictions
sudo tee /etc/systemd/system/caspida-ui.service > /dev/null << 'EOF'
[Unit]
Description=Splunk UBA UI Service (Non-Privileged)
After=network.target
Wants=network.target

[Service]
Type=forking
User=caspida
Group=caspida
Environment="JAVA_HOME=/opt/caspida/third-party/java/current"
Environment="CASPIDA_HOME=/opt/caspida"
ExecStart=/opt/caspida/bin/containers/caspida-ui start
ExecStop=/opt/caspida/bin/containers/caspida-ui stop
ExecReload=/opt/caspida/bin/containers/caspida-ui restart
PIDFile=/var/vcap/sys/run/caspida-ui/caspida-ui.pid
TimeoutStartSec=300
TimeoutStopSec=300
Restart=on-failure
RestartSec=30

# Security restrictions
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/opt/caspida /var/vcap /etc/caspida /tmp /var/tmp

# Capabilities restrictions
CapabilityBoundingSet=
AmbientCapabilities=

# System call restrictions
SystemCallFilter=@system-service
SystemCallErrorNumber=EPERM

[Install]
WantedBy=multi-user.target
EOF

# Reload systemd
sudo systemctl daemon-reload
```

4. Set up file system permissions:

```
# Ensure proper ownership for caspida user
sudo chown -R caspida:caspida /opt/caspida
sudo chown -R caspida:caspida /var/vcap
sudo chown -R caspida:caspida /etc/caspida

# Set proper permissions
sudo find /opt/caspida -type d -exec chmod 755 {} \;
sudo find /opt/caspida -type f -exec chmod 644 {} \;
sudo find /opt/caspida/bin -type f -exec chmod 755 {} \;

# Secure sensitive directories
sudo chmod 700 /etc/caspida/conf/keystore
sudo chmod 600 /etc/caspida/conf/keystore/*
```

5. Create monitoring and validation scripts:


```

# Create privilege validation script
sudo -u caspida tee /opt/caspida/bin/validate_privileges.sh > /dev/null << 'EOF'
#!/bin/bash
# Validate non-privileged UBA configuration

echo "=== UBA Non-Privileged Configuration Validation ==="

# Check user context
echo "Current user: $(whoami)"
echo "User ID: $(id)"

# Check sudo permissions
echo -e "\n=== Sudo Permissions ==="
sudo -l 2>/dev/null | head -10

# Check port bindings
echo -e "\n=== Port Bindings ==="
ss -tulpn | grep $(whoami) || echo "No ports bound to current user"

# Check process ownership
echo -e "\n=== Running Processes ==="
ps aux | grep $(whoami) | grep -v grep | head -5

# Check file permissions
echo -e "\n=== File Permissions ==="
ls -la /opt/caspida/ | head -5
ls -la /var/vcap/ | head -5

# Test service management
echo -e "\n=== Service Management Test ==="
sudo systemctl status caspida-ui.service >/dev/null 2>&1 && echo "✓ Can check service status" || echo "✗ Cannot check service status"

# Check certificate access
echo -e "\n=== Certificate Access ==="
ls -la /etc/caspida/conf/keystore/ 2>/dev/null | head -3 || echo "No keystore directory or access denied"

echo -e "\n=== Validation Complete ==="
EOF

chmod +x /opt/caspida/bin/validate_privileges.sh

```

Testing Non-Privileged Configuration

1. Validate configuration:

```

# Run validation script
sudo -u caspida /opt/caspida/bin/validate_privileges.sh

# Test service management
sudo -u caspida sudo systemctl status caspida-ui.service

# Test UBA functionality
sudo -u caspida /opt/caspida/bin/Utils/uba_health_check.sh

```

2. Performance and security verification:

```

# Check for privilege escalation attempts
sudo journalctl -u caspida-ui.service | grep -i "permission\|denied\|error" | tail -10

# Verify port binding
ss -tulpn | grep caspida

# Test UI accessibility on non-privileged ports
curl -k -I http://localhost:8080/
curl -k -I https://localhost:8443/

```

Migration Best Practices

1. Pre-migration checklist:

- ☐ Full system backup completed
- ☐ UBA services verified functional with script management
- ☐ systemd version compatibility confirmed
- ☐ Non-privileged port requirements assessed
- ☐ Security policy reviewed and approved

2. Migration validation:

- ☐ systemd services start/stop correctly
- ☐ UBA health check passes
- ☐ Web UI accessible on configured ports
- ☐ Job Manager API responding
- ☐ Data ingestion continuing normally
- ☐ No privilege escalation errors in logs

3. Rollback procedure:

```
# If migration fails, rollback to script management
sudo systemctl stop caspida-*.service
sudo systemctl disable caspida-*.service

# Remove systemd units
sudo rm /etc/systemd/system/caspida-*.service
sudo systemctl daemon-reload

# Restore original configuration
sudo -u caspida /opt/caspida/bin/Caspida start-all
```

18. Deployment Summary and Next Steps

Deployment Validation

Use the comprehensive **Splunk_UBA_Deployment_Checklist.md** document to validate all deployment tasks. This checklist covers:

- Pre-installation validation (hardware, OS, network, storage)
- Installation process verification
- Configuration validation
- Data source setup confirmation
- Security and authentication validation
- Performance and monitoring setup
- Final production readiness sign-off

Post-Deployment Activities

1. **Monitor Initial Operation:**
 - Review system health for first 24-48 hours
 - Monitor data ingestion rates and processing
 - Verify anomaly and threat detection begins
2. **Optimization and Tuning:**
 - Adjust performance parameters based on actual workload
 - Fine-tune anomaly detection thresholds
 - Optimize data retention policies
3. **User Training:**
 - Train security analysts on UBA interface
 - Provide documentation for common tasks
 - Establish escalation procedures
4. **Ongoing Maintenance:**
 - Establish regular backup verification
 - Schedule periodic security updates
 - Plan for certificate renewals
 - Monitor system capacity and plan for scaling

Key Resources

- **Splunk UBA Documentation:** Official Splunk UBA documentation on docs.splunk.com
- **Splunk Community:** answers.splunk.com for community support
- **Splunk Support:** support.splunk.com for enterprise support
- **Splunk Professional Services:** For advanced implementation assistance
- **Troubleshooting Guide:** Splunk_UBA_Troubleshooting_Guide.md for detailed issue resolution

Emergency Procedures

Keep the following information readily available:

1. **Admin Password Reset Procedure:**

```
/opt/caspida/bin/admin_password_reset.sh
```

2. **Emergency Service Restart:**

```
/opt/caspida/bin/Caspida stop-all
/opt/caspida/bin/Caspida start-all
```

3. **Health Check Command:**

```
/opt/caspida/bin/utils/uba_health_check.sh
```

4. **Log Locations:**

- UI: `/var/vcap/sys/log/caspida-ui/`
- Job Manager: `/var/vcap/sys/log/caspida-jobmanager/`
- Data Sources: `/var/vcap/sys/log/caspida-datasource/`
- System: `/var/log/caspida/`

Support Information

Version: Splunk UBA 5.4.2 on RHEL 8.10

Document Version: 1.0

Last Updated: June 2025

For technical support, gather the following information: - UBA version and build number - RHEL 8.10 version and kernel information - Deployment topology (single/distributed) - Description of issue with relevant log entries - Output of health check script

Deployment Complete! Your Splunk UBA 5.4.2 environment is now ready for production use.

Note: For comprehensive deployment validation, refer to the separate ***Splunk_UBA_Deployment_Checklist.md*** document.

Troubleshooting: For detailed troubleshooting procedures, refer to ***Splunk_UBA_Troubleshooting_Guide.md***.